Hands-on Lab: Generative AI for Data Preparation

Estimated time: 30 minutes

In practice, the initial part of a data science workflow involves cleaning and preparing data for better analysis. This part usually requires the removal of blank entries, normalization of numerical attributes, numerical interpretation of categorical variables, and so on. In this lab, you will use a generative AI model to create a Python code to perform all the required tasks on a real-world data set.

Learning objectives

In this lab, you will learn how to use generative AI for creating a Python code to:

- Handle missing values in the data set
- Correct the data type for the required data set attributes
- Perform standardization and normalization on required parameters
- Convert categorical data into numerical indicator variables

About generative AI classroom lab

▶ Click here

Notes:

1. The prompts used in this lab are for your reference only. You can create your own prompts and generate responses using generative AI.
2. Since AI-generated outputs are dynamic, you may receive different responses even though you've used the same prompt from this lab.

Code execution environment

To test the prompt-generated code, keep the Jupyter Notebook (in the link below) open in a separate tab in your web browser. The notebook has some setup instructions that you should complete now.

Jupyter-Lite Test Environment

The data set for this lab is available in the following URL:

```
URL = "https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBMDeveloperSkillsNetwork-DA0101EN-Coursera/laptop_pricing_dataset_mod1.csv
```

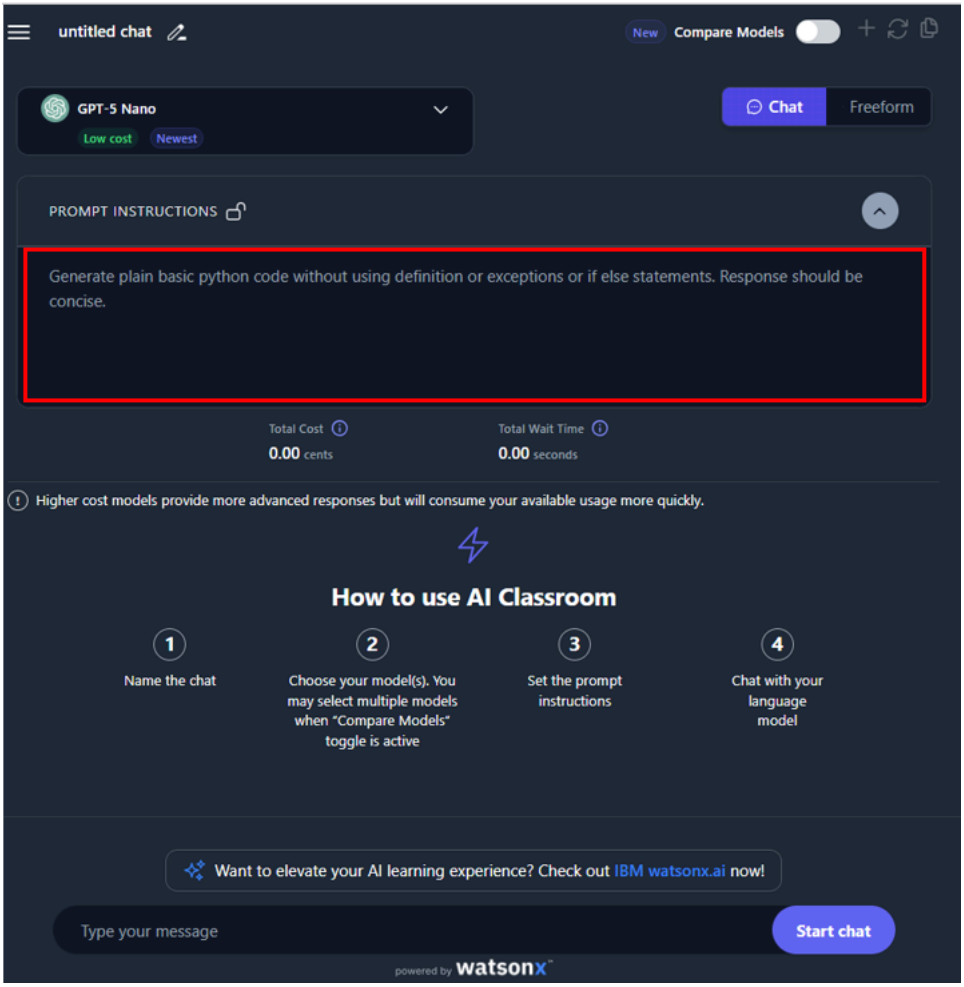
The dataset is a filtered and modified version of the Laptop Price Prediction using specifications dataset, available under the Database Contents License (DbCL) v1.0 on the Kaggle website.

Complete the setup in the Jupyter Notebook and then proceed to the following actions.

Building the prompt: Importing the data set

Now, you will start building your prompt. You can use the below to get a defined output:

“Generate plan basic python code without using definition or exceptions or if else statements. Response should be concise.”



First, you must ask the Gen AI model to generate a code to import the provided data set to a Pandas' data frame. You must specify if you are importing the data. Assuming the first row is the header, the data set headers should be available as the first row of the CSV file.

The prompt to create the code can be structured as follows.

```
Write a Python code that can perform the following tasks.
Read the CSV file, located on a given file path, into a Pandas data frame, assuming that the first rows of the file are the headers for the data.
```

Running this prompt using the GPT-5 Nano engine in the GenAI classroom should result in the following code:

```
This script reads a CSV file from the given path into a Pandas DataFrame. The first row is treated as headers by default (header=0).
import pandas as pd
# Path to the CSV file
file_path = "path/to/your/file.csv"
# Read the CSV into a DataFrame. By default, header=0, so the first row is treated as column names.
df = pd.read_csv(file_path)
# Optional: display basic information about the loaded DataFrame
print(df.head())
print('Rows:', len(df), 'Columns:', len(df.columns))
```

You may copy the code from the response and try it on the Jupyter Notebook with appropriate modifications in the file path.

Building the prompt: Handle missing data

You can now ask the Generative AI model to generate a script to handle the missing data.

First, use the model to identify the attributes with missing data.

For this, you may run the following query.

```
Write a Python code that identifies the columns with missing values in a pandas data frame and gives missing value counts per column.
```

You can use the relevant part of the code generated for your task.

```
This script reads a CSV into a Pandas DataFrame, computes which columns have any missing values using isna().any(), and prints the list of such colum
import pandas as pd
# Path to the CSV file
file_path = "path/to/your/file.csv"
# Read the CSV into a DataFrame (first row treated as headers by default)
df = pd.read_csv(file_path)
# Identify columns that contain any missing values
cols_with_missing = df.columns[df.isna().any()].tolist()
# Output the list of columns with missing values
print("Columns with missing values:", cols_with_missing)
# Optional: display missing value counts per column
missing_counts = df.isna().sum()
print("Missing values per column:")
print(missing_counts)
```

Once run, you will see that the two attributes with missing values are "Screen_Size_cm", a categorical variable, and "Weight_kg", a continuous variable.

Now, you need to replace the missing values with appropriate values. The following are the rules for this:

1. Missing entries in columns containing categorical values need to be replaced with the most frequent entries
2. Missing entries in columns with continuous data need to be replaced with the mean value of the column
3. If a value is missing in the target column, you may need to drop that row

You can build a prompt for this, as shown below.

```
Write a Python code to replace the missing values in a pandas data frame, per the following guidelines.
1. For a categorical attribute "Screen_Size_cm", replace the missing values with the most frequent value in the column.
2. For a continuous value attribute "Weight_kg", replace the missing values with the mean value of the entries in the column.
```

The prompt will generate a code. You can extract the required statements, as shown below.

```
This script fills missing values in Screen_Size_cm with the column's most frequent value (mode) and in Weight_kg with the column's mean, using a sing
import pandas as pd
# df is the existing DataFrame
# 1) Replace missing values in the categorical column with its most frequent value
screen_size_mode = df['Screen_Size_cm'].mode().iloc[0]
# 2) Replace missing values in the continuous column with its mean value
weight_kg_mean = df['Weight_kg'].mean()
# Apply the replacements
df = df.fillna({
    'Screen_Size_cm': screen_size_mode,
    'Weight_kg': weight_kg_mean
})
```

Building the prompt: Modify data type

Further, you should update both attributes' data type to floating values.

You should see a similar response to the following prompt:

```
Write a Python code snippet to change the data type of the attributes "Screen_Size_cm" and "Weight_kg" of a data frame to float.
```

You should see a code as shown below:

```
Convert the two specified DataFrame columns to float dtype using astype for numeric operations. Assumes the columns exist and contain values that can
df['Screen_Size_cm'] = df['Screen_Size_cm'].astype(float)
df['Weight_kg'] = df['Weight_kg'].astype(float)
```

You can use the relevant part of the code in your script.

Once run, the code data attributes will have the correct data type.

Building the prompt: Standardization and Normalization

You may notice that the parameter `Screen_Size_cm` represents the screen size in centimeters instead of the standard unit, which is inches. Also notice that the parameter `Weight_kg` would be better expressed in pounds.

You can ask generative AI to create a code to convert these parameters without having to mention the conversion rates.

You can build a prompt for this, as shown below.

```
Write a Python code to modify the contents under the following attributes of the data frame as required.
1. Data under 'Screen_Size_cm' is assumed to be in centimeters. Convert this data into inches. Modify the name of the attribute to 'Screen_Size_inch'
2. Data under 'Weight_kg' is assumed to be in kilograms. Convert this data into pounds. Modify the name of the attribute to 'Weight_pounds'.
```

The expected output should look as shown below.

```
This code converts Screen_Size_cm to inches by dividing by 2.54, creates a new Screen_Size_inch column, and removes the original Screen_Size_cm colum
# Convert Screen_Size_cm (cm) to Screen_Size_inch and rename the column
# Create a new column with inches and drop the original centimeter column
df['Screen_Size_inch'] = df['Screen_Size_cm'] / 2.54
df = df.drop(columns=['Screen_Size_cm'])
# Convert Weight_kg (kg) to Weight_pounds and rename the column
# Create a new column with pounds and drop the original kilogram column
df['Weight_pounds'] = df['Weight_kg'] * 2.2046226218
df = df.drop(columns=['Weight_kg'])
```

You can use the relevant part of the code in your script.

It may also be required to normalize the data under some attributes. Since there are many normalization forms, mentioning the exact needs and tasks is important. Also, you can save the normalized data as a new attribute or change the original attribute. You need to ensure that all the details of the prompt are clear.

For example, let us assume that the data under "CPU_frequency" needs to be normalized w.r.t. the max value under the attribute. You need the changes to be reflected directly under the attribute instead of creating a new attribute.

You can ask Generative AI to generate a script that does this for you.

You can build a prompt for this, as shown below.

```
Write a Python code to normalize the content under the attribute "CPU_frequency" in a data frame df concerning its maximum value. Make changes to the
```

This should generate the response as shown below.

```
Normalize the CPU_frequency column by its maximum value in place, replacing the original data without creating a new attribute.
df['CPU_frequency'] /= df['CPU_frequency'].max()
```

Use the relevant part of the code in your script.

Building the prompt: Categorical to numerical

For predictive modeling, the categorical variables are not usable currently. So, you must convert the important categorical variables into indicator numerical variables. Indicator variables are typically new attributes, with content being 1 for the indicated category and 0 for all others. Once you create the indicator variables, you may drop the original attribute.

For example, assume that attribute `Screen` needs to be converted into individual indicator variables for each entry. Once done, the attribute `Screen` needs to be dropped.

You can build a prompt for this, as shown below.

```
Write a Python code to perform the following tasks.
1. Convert a data frame df attribute "Screen", into indicator variables, saved as df1, with the naming convention "Screen_<unique value of the attrib
2. Append df1 into the original data frame df.
3. Drop the original attribute from the data frame df.
```

The expected response is as shown below.

```
This code performs one-hot encoding on the 'Screen' column, stores the resulting indicator columns in df1, appends them to the original DataFrame, ar
# 1) Convert 'Screen' into indicator variables named 'Screen_<value>'
df1 = pd.get_dummies(df['Screen'], prefix='Screen')
# 2) Append the indicator variables to the original DataFrame
df = df.join(df1)
# 3) Drop the original 'Screen' column from the DataFrame
df.drop(columns=['Screen'], inplace=True)
```

You can use the relevant part in your code.

Practice problems

1. Create a prompt to generate a Python code that converts the values under `Price` from USD to Euros.
2. Modify the normalization prompt to perform min-max normalization on the `CPU_frequency` parameter.

Conclusion

Congratulations! You have completed the lab on data preparation.

With this, you have learned the process of using generative AI to create Python codes that can:

1. Handle missing values in the data frame
2. Modify the attribute data types
3. Perform attribute transformations like standardization and normalization
4. Convert categorical attributes into indicator variable attributes.

Author(s)

Abhishek Gagneja